

Análise Sintática Top-Down

Fatoração à Esquerda, Gramáticas $LL(1)$ e Conjuntos $FIRST/FOLLOW$

Compiladores

Fatoração à Esquerda

- ▶ **Utilidade:** Útil para analisadores sintáticos *top-down*.
- ▶ **Situação:** Temos duas ou mais alternativas de produção para um não-terminal A :

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$

- ▶ **Problema:** Qual produção escolher ao analisar a entrada?
- ▶ Podemos reescrever as produções para adiar a escolha até que tokens suficientes tenham sido lidos para permitir a escolha correta.

Fatoração à Esquerda

Exemplo

$\text{stmt} \rightarrow \boxed{\text{if expr then stmt}} \mid \boxed{\text{if expr then stmt}} \text{ else stmt}$

Ao ler o token **if**, não sabemos qual produção escolher até a substituição do símbolo não-terminal **stmt**.

Regra e Algoritmo de Fatoração

Regra Geral

Se $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ são duas produções de A , e a entrada começa com o mesmo prefixo α ($\alpha \neq \varepsilon$), adiamos a decisão reescrevendo para:

Regra e Algoritmo de Fatoração

Regra Geral

Se $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ são duas produções de A , e a entrada começa com o mesmo prefixo α ($\alpha \neq \varepsilon$), adiamos a decisão reescrevendo para:

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 \mid \beta_2$$

Regra e Algoritmo de Fatoração

Algoritmo para Fatorar uma Gramática:

1. Para um não-terminal A , encontre o prefixo comum mais longo α entre duas ou mais produções:

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \cdots \mid \alpha\beta_n \mid \gamma$$

(onde γ representa produções sem o prefixo comum α).

2. Substitua essas produções por:

$$A \rightarrow \alpha A' \mid \gamma$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

3. Repita o processo até que não existam mais produções com prefixo comum.

Exemplo de Fatoração

Considere a gramática com o prefixo comum $\alpha = \mathbf{if\ } E \mathbf{\ then\ } S$:

$S \rightarrow \mathbf{if\ } E \mathbf{\ then\ } S \mid \mathbf{if\ } E \mathbf{\ then\ } S \mathbf{\ else\ } S \mid \mathbf{other}$

$E \rightarrow \mathbf{b}$

Exemplo de Fatoração

Considere a gramática com o prefixo comum $\alpha = \mathbf{if\ } E \mathbf{\ then\ } S$:

$$S \rightarrow \mathbf{if\ } E \mathbf{\ then\ } S \mid \mathbf{if\ } E \mathbf{\ then\ } S \mathbf{\ else\ } S \mid \mathbf{other}$$
$$E \rightarrow \mathbf{b}$$

Aplicando a fatoração à esquerda:

$$\beta_1 = \varepsilon \quad \text{e} \quad \beta_2 = \mathbf{else\ } S$$

Gramática Fatorada à Esquerda:

$$S \rightarrow \mathbf{if\ } E \mathbf{\ then\ } S S' \mid \mathbf{other}$$
$$S' \rightarrow \varepsilon \mid \mathbf{else\ } S$$
$$E \rightarrow b$$

Analizador Sintático Top-Down

- ▶ Constrói (explícita ou implicitamente) uma árvore de derivação para a entrada.
- ▶ A árvore é construída da **raiz para as folhas**.
- ▶ Implementa derivações mais à esquerda.

Problema Chave da Análise Top-Down

Estando num passo da análise e considerando o não-terminal A mais à esquerda na sentença atual, **qual produção de A devemos aplicar?**

Analísadores Preditivos e Recursivo-Descendentes

Analísadores Sintáticos Preditivos:

- ▶ Escolhem a produção de A a ser aplicada olhando para um número mínimo k de tokens da entrada (*lookahead*).
- ▶ Evitam *backtracking* (retrocesso).

Implementação por Reconhecedores Recursivos:

- ▶ Cada não-terminal A possui uma função associada.
- ▶ A função tenta encontrar uma produção de A que consome tokens da entrada.
- ▶ Chamada do símbolo inicial S : se consome toda a entrada, esta é bem-formada; caso contrário, é mal-formada.

Gramáticas $LL(k)$

Definição

Dada uma gramática G , ela é $LL(k)$ se é possível construir um analisador sintático preditivo que lê a entrada da esquerda para a direita, faz derivação mais à esquerda e escolhe a produção correta olhando para **no máximo k tokens** de entrada.

Significado da sigla $LL(k)$:

- ▶ **L**: Leitura da entrada da esquerda para a direita (*Left-to-right scan*).
- ▶ **L**: Derivação mais à esquerda (*Leftmost derivation*).
- ▶ **k**: Número de tokens lidos da entrada (*lookahead*).

Conjuntos *FIRST* e *FOLLOW*

Vamos construir conjuntos *FIRST* e *FOLLOW*.

Motivação - São utilizados na

1. Construção de analisadores *LL*(1) (*top-down*) e *LR* (*bottom-up*).
2. Demonstração de que uma gramática é *LL*(1).

Conjuntos *FIRST*

Conjunto $FIRST(\alpha)$ sendo α uma sequência de símbolos

É o conjunto de terminais que iniciam cadeias derivadas a partir de α . Se $\alpha \xRightarrow{*} \varepsilon$, então $\varepsilon \in FIRST(\alpha)$.

Regras para Cálculo do *FIRST*

Para qualquer símbolo X :

1. Se X é terminal, $FIRST(X) = \{X\}$.
2. Se X é não-terminal e $X \rightarrow Y_1 Y_2 \dots Y_k$:
 - ▶ Adicione $FIRST(Y_1) \setminus \{\varepsilon\}$ a $FIRST(X)$.
 - ▶ Se $\varepsilon \in FIRST(Y_1)$, adicione $FIRST(Y_2) \setminus \{\varepsilon\}$, e assim por diante.
 - ▶ Se todos $Y_1, \dots, Y_k$ contiverem ε , adicione ε a $FIRST(X)$.
3. Se $X \rightarrow \varepsilon$ é uma produção, adicione ε a $FIRST(X)$.

Regras para Cálculo do *FIRST*

Para qualquer símbolo X :

1. Se X é terminal, $FIRST(X) = \{X\}$.
2. Se X é não-terminal e $X \rightarrow Y_1 Y_2 \dots Y_k$:
 - ▶ Adicione $FIRST(Y_1) \setminus \{\varepsilon\}$ a $FIRST(X)$.
 - ▶ Se $\varepsilon \in FIRST(Y_1)$, adicione $FIRST(Y_2) \setminus \{\varepsilon\}$, e assim por diante.
 - ▶ Se todos $Y_1, \dots, Y_k$ contiverem ε , adicione ε a $FIRST(X)$.
3. Se $X \rightarrow \varepsilon$ é uma produção, adicione ε a $FIRST(X)$.

E como calcular $FIRST(\alpha)$ sendo α uma sequência de símbolos da gramática?

Algoritmo: *FIRST* de uma Sequência ($\alpha = X_1X_2 \dots X_n$)

Para determinar $FIRST(X_1X_2 \dots X_n)$:

1. **Inicialização:** $FIRST(X_1X_2 \dots X_n) = \emptyset$
2. **Varredura da Sequência (da esquerda para a direita):**
Para cada i de 1 até n :
 - ▶ Adicione $FIRST(X_i) \setminus \{\varepsilon\}$ a $FIRST(X_1X_2 \dots X_n)$.
 - ▶ Se $\varepsilon \notin FIRST(X_i)$:
 - ▶ **Pare a varredura!** (Não analise os símbolos seguintes e encerre o algoritmo).
3. **Inclusão de ε (Caso Especial):**
Se a varredura chegou ao fim (ou seja, $\varepsilon \in FIRST(X_i)$ para **todos** os $i = 1, \dots, n$):
 - ▶ Adicione ε a $FIRST(X_1X_2 \dots X_n)$.

* *Nota:* Se $\alpha = \varepsilon$ (cadeia vazia), $FIRST(\varepsilon) = \{\varepsilon\}$.

Conjuntos *FOLLOW*

Conjunto $FOLLOW(A)$ sendo A um não-terminal

É o conjunto de terminais a que podem aparecer imediatamente à direita de A em alguma forma sentencial:

$$S \xRightarrow{*} \alpha A a \beta$$

Se A pode ser o símbolo mais à direita, o marcador de fim de entrada $\$$ pertence a $FOLLOW(A)$.

Regras para Cálculo do *FOLLOW*

Para cada não-terminal B :

1. Adicione $\$$ ao $FOLLOW(B)$, se B é o símbolo inicial da gramática ($\$$ indica o fim da entrada).
2. Se existe uma produção $A \rightarrow \alpha B \beta$, então tudo em $FIRST(\beta)$ (exceto ε) está em $FOLLOW(B)$.
3. Se existe uma produção $A \rightarrow \alpha B$, ou $A \rightarrow \alpha B \beta$ onde $FIRST(\beta)$ contém ε , então tudo em $FOLLOW(A)$ está contido em $FOLLOW(B)$.

* *Nota:* ε **nunca** estará em qualquer conjunto *FOLLOW*.

Exemplo: Cálculo de *FIRST* e *FOLLOW*

Gramática:

$$\begin{aligned}E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{id}\end{aligned}$$

Conjuntos *FIRST*:

- ▶ $FIRST(E) = FIRST(T) = FIRST(F) = \{ (, \text{id} \}$
- ▶ $FIRST(E') = \{ +, \varepsilon \}$
- ▶ $FIRST(T') = \{ *, \varepsilon \}$

Conjuntos *FOLLOW*:

- ▶ $FOLLOW(E) = FOLLOW(E') = \{), \$ \}$
- ▶ $FOLLOW(T) = FOLLOW(T') = \{ +,), \$ \}$
- ▶ $FOLLOW(F) = \{ *, +,), \$ \}$

Condição para uma Gramática ser $LL(1)$

Teorema

Uma gramática G é $LL(1)$ se e somente se para quaisquer duas produções distintas $A \rightarrow \alpha \mid \beta$ vale que:

1. $FIRST(\alpha) \cap FIRST(\beta) = \emptyset$; e
2. Se $\beta \xRightarrow{*} \varepsilon$, então $FIRST(\alpha) \cap FOLLOW(A) = \emptyset$ (ou o caso simétrico se $\alpha \xRightarrow{*} \varepsilon$).

Tabela de Análise $M[A, a]$: Contém no máximo uma produção por célula para cada combinação de não-terminal A e terminal a . Se não houver produção na célula, temos um **erro sintático**.

Tabela de Reconhecimento Sintático

- ▶ Linhas indexadas por não-terminais.
- ▶ Colunas indexadas por tokens.

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

Algoritmo: Construção da Tabela de Reconhecimento Sintático

Dada uma gramática G , preenchamos a tabela $M[A, a]$ executando os seguintes passos para cada produção $A \rightarrow \alpha$:

1. **Mapeamento via $FIRST$:**

- ▶ Para cada terminal $a \in FIRST(\alpha)$, adicione $A \rightarrow \alpha$ em $M[A, a]$.

2. **Mapeamento via $FOLLOW$ (se a regra derivar ε):**

- ▶ Se $\varepsilon \in FIRST(\alpha)$, para cada símbolo $b \in FOLLOW(A)$ (incluindo o fim da entrada $\$$), adicione $A \rightarrow \alpha$ em $M[A, b]$.

Algoritmo: Construção da Tabela de Reconhecimento Sintático

Dada uma gramática G , preenchamos a tabela $M[A, a]$ executando os seguintes passos para cada produção $A \rightarrow \alpha$:

1. **Mapeamento via $FIRST$:**

- ▶ Para cada terminal $a \in FIRST(\alpha)$, adicione $A \rightarrow \alpha$ em $M[A, a]$.

2. **Mapeamento via $FOLLOW$ (se a regra derivar ε):**

- ▶ Se $\varepsilon \in FIRST(\alpha)$, para cada símbolo $b \in FOLLOW(A)$ (incluindo o fim da entrada $\$$), adicione $A \rightarrow \alpha$ em $M[A, b]$.

Propriedade Fundamental de $LL(1)$

- ▶ Se qualquer célula $M[A, a]$ contiver **mais de uma produção**, a gramática **não é $LL(1)$** (ocorre conflito).
- ▶ Posições da tabela vazias indicam **erros sintáticos**.

Algoritmo de Análise Sintática com Pilha

► **Entrada:** Cadeia $w\$$, onde $\$$ marca o fim da entrada.

1. Inicialize uma pilha com o símbolo de fim $\$$ e o símbolo inicial S no topo.
2. Defina $i = 1$ (início da entrada) e seja X o topo da pilha.
3. Enquanto $X \neq \$$ faça:
 - 3.1 Se $X = w[i]$, então desempilha X e avança i .
 - 3.2 Senão se X é terminal, então acuse erro e pare.
 - 3.3 Senão se $M[X, w[i]]$ está vazia, então acuse erro e pare.
 - 3.4 Senão se $M[X, w[i]] = X \rightarrow Y_1 Y_2 \dots Y_k$, então
 - 3.4.1 Imprima a produção $X \rightarrow Y_1 Y_2 \dots Y_k$.
 - 3.4.2 Desempilha X .
 - 3.4.3 Empilha $Y_k Y_{k-1} \dots Y_1$.
 - 3.5 Seja X o topo da pilha.